

BSE SPECIALISATION ADVISOR

Project Documentation

Bibi Shania Tinkouree BSc (Hons) Software Engineering, Year 1

Frontend Web Development Summative Assessment (June October 2026)

1. Project Overview

The BSE Specialisation Advisor is a web application that helps Bachelor of Software Engineering students discover which of the four BSE specialisation pathways Low-Level Programming, AR / VR, Full-Stack Web Development and Machine Learning best matches their strengths, interests and goals. The student completes a short 10-question quiz; every answer scores points towards one of the four categories; and a personalised results page then shows the student's top specialisation, a full category breakdown, an animated radar chart and next-step recommendations.

The project is a static multi-page site built entirely with HTML5, CSS3 and vanilla JavaScript. It uses no frameworks, no build tools and no backend, so it runs directly in any modern browser and can be hosted free on GitHub Pages. All data lives in the browser's sessionStorage, which keeps the whole experience self-contained.

The site welcomes students from every intake level (Year 1-3). It uses an institutional indigo and amber palette, a consistent layout across all four pages, and keyboard-friendly interactions throughout.

2. Pages and Site Structure

The site consists of four pages connected by a shared header navigation (with an active-page highlight and a mobile hamburger menu) and a shared footer.

index.html Home. The hero introduces the tool and the four pathways. Four preview cards explain each specialisation, and a "Before we begin" form captures the student's profile (first name, last name, student ID, institutional email, phone number and intake level) with real-time validation before the quiz starts.

quiz.html Quiz. Rebuilds one question card at a time from a data-driven question set, with a progress bar, a question counter and a live 5-minute countdown.

Includes the three interactive media question types (image hotspot, audio, video).

results.html Results. Reads the saved quiz payload and renders the personalised outcome: a hero announcing the top specialisation, animated score bars, a pure-Canvas radar chart, a ranked narrative and next-step recommendations, plus actions to retake the quiz, contact the team or update details.

contact.html Contact. A feedback form with an interactive star rating that reuses the same live validation engine; on submit it shows an inline success banner (no server round trip).

3. Quiz Content and Question Types

The quiz asks 10 questions, each mapping to one specialisation. Seven are standard multiple-choice questions and three are interactive media questions. The questions

are deliberately scenario-based a free afternoon, a debugging instinct, a dream project, a toolbox so students answer from instinct rather than technical knowledge.

Q1 Q4 (choice): free-afternoon preference, debugging instinct, dream final-year project and first skill to sharpen.

Q5 (image hotspot): tap the piece of tech in the scene that calls to you.

Q6 (audio): listen to a clip with the custom player and pick the developer vibe it sounds like.

Q7 (video): watch a startup demo that pauses at a key moment, then decide what to debug first.

Q8 Q10 (choice): favourite toolbox, natural team role and dream product to ship.

Each question type renders its own interface, but they all feed the same scoring engine, so the format does not change how results are computed.

4. Interactive Media Features

Image hotspot (Q5)

The hotspot question shows an original hand-drawn SVG "tech hub" scene containing four objects a circuit board (Low-Level), a laptop with code (Full-Stack Web), a VR headset (AR/VR) and a robot (Machine Learning). Each object is wrapped in a hotspot region that records its category when selected.

Invisible 72 px hit circles make the whole object clickable, not just a small dot or ring.

A dashed ring and core dot light up on hover and stay lit on selection; picking a new region cleanly deselects the previous one.

Regions are keyboard-accessible (Tab to focus, Enter or Space to select) and carry descriptive aria-labels.

A confirm button fades in once a region is chosen, shows the awarded points, and then advances the quiz.

Audio question (Q6)

Plays assets/audio-prompt.wav through an HTML5 audio element driven by custom buttons

Play/Pause and Replay instead of the default controls. A status line reports Ready to play / Playing / Paused / Finished, and when the clip ends the play button changes to Replay.

Video question (Q7)

Plays the 12-second assets/video-scenario.webm clip. A timeupdate listener auto-pauses the video at 5 seconds and reveals the decision panel with the four options; a Resume button lets the student watch the rest before answering.

5. Scoring System

Answers are scored into the four categories, and the winning specialisation is simply the category with the highest total. The scoring combines base points, a speed bonus and a streak multiplier so the result reflects both preference and consistency.

Base points: 10 points per answered question.

Speed bonus: +5 points when the answer is given within 8 seconds, +3 within 15 seconds, otherwise +1 rewarding quick, instinctive choices.

Streak multiplier: consecutive answers in the same category grow a multiplier from

1.0× to a cap of 1.5×(+0.1 per consecutive pick, capping after 6 in a row)

rewarding a strong, consistent preference.

Question points are computed as $\text{round}((10 + \text{bonus}) \times \text{multiplier})$.

After every answer a toast shows the breakdown "+X pts speed bonus +Y streak
XZ" for 3 seconds, then the quiz advances after a short 1.6-second pause.

Each answer is logged with its question number, type, category, bonus, multiplier and points, and this log drives the results-page narrative.

6. Timer and Timeout Handling

The quiz runs on a 5-minute (300-second) countdown driven by `setInterval`. Each tick updates the on-screen time and the progress bar, so the student always knows how much time is left.

With 30 seconds or less left, the timer badge turns red and pulses as a visual warning.

If time expires, the quiz locks all controls (pointer events disabled), shows a "Time's up!" overlay explaining that current answers were recorded, and after 1.4 seconds auto-submits and redirects to the results page.

The results page notes when a run timed out: "You ran out of time answers recorded up to the final question."

7. Data, State and Validation

Student profile

The landing form saves the student profile to `sessionStorage` under the key `bse-student`: first name, last name, a combined name, student ID, lowercased institutional email, phone, intake level and a timestamp. The quiz reads the profile to greet the student by name; if it is missing, the quiz politely blocks with a "Go to home page" link instead of crashing.

Quiz results

On finishing, the quiz saves a `bse-results` payload (scores, the per-question log, time used, a timed-out flag, completion time and the student profile) and navigates to `results.html`. Opening results without data shows a friendly "No results yet" state with a link to the quiz.

`sessionStorage` is used rather than `localStorage` deliberately, so a fresh browser session always starts at the landing page and never shows stale results.

Validation engine (js/shared.js)

A single shared validation engine powers every form. Each field carries a `data-validate` attribute; a central `RULES` object maps every field name to a regular expression and an error message; input and blur listeners validate live; and one render routine toggles the `.is-valid` / `.is-invalid` classes and shows an inline error message. There are no `alert()` popups anywhere.

Name: 2 50 letters only (accents, spaces, apostrophes and hyphens allowed; no double separators, numbers or symbols).

Student ID: the BSE pattern BSE followed by 4 6 digits (e.g. BSE25013).

Institutional email: local part with letters, digits and `.` `_` - separators, on the `@alustudent.com` domain (e.g. `student.id@alustudent.com`).

Phone: optional +230 country code plus 8 local digits starting with 2 9 (e.g.

+23058123456 or 58123456).

Message (contact page): 20 1000 characters.

The intake-level dropdown uses a plain required check; if a submit is attempted with errors, focus moves to the first invalid field.

8. Results Page Rendering

The results page builds the entire view from the stored payload:

Ranking: the four categories are sorted by score; the hero announces the top specialisation in its own colour, followed by the finishing time (or a timeout note).

Score bars: each category fills from zero with a CSS width transition, and the numbers count up with a `requestAnimationFrame` ease-out loop over about one second.

Narrative: every category gets a personalised description plus rank-based copy (Your strongest signal , A very close second , etc.).

Recommendations: the top two pathways get next-step action cards with module suggestions, activities and target roles.

Radar chart: a spider chart drawn with the raw Canvas 2D API no charting library. It scales by `devicePixelRatio` for crisp text, draws concentric grid rings at 25/50/75/100%, axis spokes, a data polygon filled with a gradient tinted from the winning colour, vertex dots, and labels that flip upright in the lower half. A `requestAnimationFrame` sweep reveals the polygon over 1.2 seconds.

Confetti: when the chart completes, roughly 70 particles burst from the winning axis with randomised velocities, sizes and spin, fall under gravity, and fade out over about 3 seconds.

9. Design and Styling

The visual style is defined by CSS custom properties in `css/style.css`, kept in one place so colours, radii and shadows stay consistent across pages.

Institutional indigo primary (#2743a8) with a darker hover shade and a pale tint, plus a warm amber accent (#f59e0b).

Four category colours Low-Level #f59e0b, AR/VR #8b5cf6, Full-Stack Web #0ea5e9, Machine Learning #10b981 reused across the home cards, the quiz hotspots, the score bars, the radar chart and the narrative.

Neutral surface/ink/muted/line palette, 10 14 px corner radii, a small shadow scale and one shared .22s cubic-bezier motion curve for smooth transitions.

Typography uses the system font stack (Segoe UI, system-ui,), so no webfont is loaded.

Layout is responsive with breakpoints at 860 px and 600 px; the header collapses into a hamburger menu and card grids reflow into single columns.

Icons and the logo are inline SVGs (a compass inside a chip), so no external icon font is required.

10. Accessibility

Semantic landmarks (header, main, footer), real `<button>` and `<label>` elements, and aria-live regions for the question card, the timer and the inline validation messages.

The quiz progress bar is exposed with `role="progressbar"`.

Hotspot regions are focusable, answer on Enter or Space, and carry descriptive aria-labels; the star rating is fully controllable with the Left and Right arrow keys.

Visible focus rings everywhere, including on the hotspot hit areas; decorative icons are marked aria-hidden.

Colour choices keep text readable on the pale background, and on an invalid submit the browser focus jumps to the first offending field.

11. Testing

The application was tested manually in the browser and with an automated integration suite.

Manual testing: every question type, the timer and timeout flow, the validation states and the results page were exercised in a real browser across the responsive breakpoints.

Automated testing: a Node.js + jsdom harness loads the real HTML and JavaScript and verifies the validation engine (every regex rule), the quiz flow (answering, scoring, bonuses, multipliers, timeout handling), hotspot selection and the results renderer. The main suite runs 32 assertions and the hotspot suite 19 all passing.

Bugs found and fixed through testing: (1) selecting a new hotspot left the previous one highlighted the deselection loop queried the template fragment after it had been appended; fixed by snapshotting the regions before insertion. (2) The points toast was invisible because it was removed after 500 ms; fixed by rendering it inside the card with a longer advance pause. (3) Hotspot click targets were tiny only the ring stroke was clickable; fixed with 72 px invisible hit circles plus a visible focus ring.

12. Deployment

Because the site is a collection of static files, deployment is simply a matter of publishing the repository to GitHub Pages no build step and no server configuration.

The repository was made public on GitHub.

GitHub Pages was enabled from Settings > Pages > Deploy from a branch (main / root).

The deployed site behaves identically to the local copy since there is no backend.

Repository

GitHub repository: https://github.com/shania178/bse_specialization_advisor

Live application

GitHub Pages: https://shania178.github.io/bse_specialization_advisor/

13. Media Asset Attributions

All media assets used in this project were generated programmatically for the assignment and are free of third-party copyright. No external images, icons, audio or video were used; the SVG hotspot scene on the quiz page is original vector artwork.

assets/audio-prompt.wav original retro-tech audio clip synthesised with a Python script using only the standard library wave module (square and sine tone synthesis).

assets/video-scenario.webm original 12-second scenario clip rendered with GStreamer (videotestsrc ball pattern + text overlay), encoded as VP8 video with Opus audio and muxed into WebM.

14. AI Usage Disclosure

14.1 How I used AI

In line with the assessment brief, I used generative AI tools as a supplementary learning aid during development, and I am disclosing that use here. I did not ask the AI to build the project for me: the site structure, the quiz content, the design and the layout are my own work. I used AI mainly for the more complex and specialised parts of the code, to check my approach, to understand a technique, or to help me get unstuck on a bug. For each AI-assisted piece I followed the same process: I explained what I wanted to achieve, read and understood the code I received, integrated it into my project, and then tested it in the browser to make sure it worked. When I found a problem, I went back to the AI for help and re-tested until it was correct.

14.2 Where AI helped me in the project

Image hotspot question AI helped me set up the SVG overlay and the invisible hit areas that make each object in the scene clickable, and to debug why clicking a new hotspot left the previous one still selected.

Audio question AI helped me wire the HTML5 audio element into custom play, pause and replay buttons with a progress display instead of the default controls.

Video question AI helped me implement the listener that pauses the video at the right moment and reveals the decision panel.

Countdown timer AI helped me build the setInterval/clearInterval countdown, the low-time warning, the lock screen and the automatic submit when time runs out.

Scoring AI helped me add speed bonuses and a streak multiplier to the multi-category scoring and show a points breakdown after each answer.

Results page AI helped me draw the radar chart with plain Canvas 2D (no charting library) and build the confetti animation.

Validation AI helped me work out the regular expressions for the institutional email, phone number, student ID and name fields.

Media assets AI helped me write the scripts that generated the audio and video files (a Python script for the WAV clip and a GStreamer command for the scenario video).

14.3 Verification

Every AI-assisted piece of code was reviewed, tested and understood before I included it. I ran the site at each step and used automated integration tests to check the whole quiz flow end to end. Bugs I found during testing such as the hotspot multi-selection issue and a toast message that was not visible were fixed and re-tested. The final decisions on design, content and code remain my own, and I am able to explain how every component of this project works.

15. References (APA 7th Edition)

opencode. (2026). opencode AI coding assistant [Computer software]. <https://opencode.ai>

GitHub. (n.d.). GitHub Pages. <https://docs.github.com/en/pages>

GStreamer Project. (n.d.). GStreamer: open source multimedia framework. <https://gstreamer.freedesktop.org/>

MDN Web Docs. (n.d.). Canvas API. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

MDN Web Docs. (n.d.). HTMLMediaElement: timeupdate event. https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement/timeupdate_event

MDN Web Docs. (n.d.). JavaScript guide: regular expressions. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

Python Software Foundation. (n.d.). wave read and write WAV files. <https://docs.python.org/3/library/wave.html>